BUSINESS REQUIREMENTS DOCUMENT

Aegis VPN

A self-hosted WireGuard consumer VPN — owned infrastructure, customer-held keys, no reseller in the middle.

Product	Aegis VPN — a self-hosted WireGuard consumer VPN
Document	Business Requirements Document (BRD), v1.0
Date	2026-09-23
Status	Baseline — describes the delivered MVP and the requirements still open
Owner	Product / Engineering (Hyperlink Infosystem)
Repository	`aegis-vpn` — `backend/` (NestJS API + node agent), `app/` (Flutter client), `ops/`, `docs/`

Prepared by Product & Engineering · Hyperlink Infosystem · Confidential — internal and stakeholder distribution

Contents

1 Executive summary 3

2 Business objectives 4

3 Scope 5

4 Stakeholders 6

5 User personas 7

6 Business requirements 8

7 Functional requirements as delivered 10

8 Non-functional requirements 12

9 Data requirements 13

10 Assumptions, constraints and dependencies 14

11 Risks 15

12 Release gates and acceptance criteria 16

13 Traceability 17

14 Glossary 19

1. Executive summary

Aegis VPN is a consumer virtual private network built end to end rather than resold. The company operates its own exit nodes, issues its own WireGuard peers, and ships its own Android client. There is no upstream VPN provider, no white-label SDK and no third party between the customer and the exit point.

The commercial premise is that the two things customers actually pay a VPN for — a different exit IP and a provider that cannot read their traffic — are only credible when the provider owns the infrastructure and cannot technically decrypt the tunnel. Aegis is architected so that the second claim is a structural fact: **the device generates its own WireGuard key pair and only ever uploads the public half.** The platform has no copy of any customer's private key and therefore cannot decrypt a session even under compulsion.

The MVP is delivered and running. Two exit nodes are live (Mumbai and Frankfurt), the API programs both through a node-scoped control plane, and the Flutter Android client completes handshakes against the live fleet. Monetisation is modelled — a 24-hour trial followed by a monthly or yearly plan — but the checkout is deliberately a stub that takes no money, pending selection of a payment provider. iOS is deferred by decision, not by oversight.

This document states the business requirements the delivered system satisfies, the requirements it knowingly does not yet satisfy, and the decisions and risks a commercial launch depends on.

---

2. Business objectives

#	Objective	Success measure
BO-1	Operate a consumer VPN on owned infrastructure with no reseller in the chain	100% of customer traffic exits nodes the company provisions and controls
BO-2	Make “we cannot read your traffic” a technical fact rather than a policy promise	Zero customer private keys present in any company system, by design and verifiable in code
BO-3	Ship a client that a non-technical customer can use without configuration	Install → sign up → connected in under 60 seconds, no config files, no manual key handling
BO-4	Support more than one country from a single control plane	≥2 countries selectable in-app; a peer issued for a country is programmed on that country’s node
BO-5	Establish a subscription revenue model with a low-friction trial	Trial → paywall → subscription walkable end to end; conversion measurable once payments are live
BO-6	Keep per-customer operating cost low enough for a sub-₹200/month price point	Fleet cost per active device within the margin implied by ₹149/month
BO-7	Build the fleet so a compromised component does not become a compromised fleet	A breach of the API, or of any single node, does not yield control of other nodes or of customer traffic

3. Scope

3.1 In scope (current release)

- Android mobile client (Flutter) with account, connection, locations, devices, settings.
- Public REST API (NestJS) for accounts, sessions, nodes, devices and entitlement.
- Node agent for programming WireGuard peers on exit nodes the API does not run on.
- Exit-node stack: WireGuard, nftables egress policy, Unbound resolver, Blocky filtering.
- Provisioning and operations scripts and runbooks for standing up a new node.
- Trial and subscription entitlement model, with a stubbed checkout.

3.2 Out of scope (this release, by decision)

Item	Reason
iOS client (Network Extension)	Requires a paid Apple organisation account; Dart layer is already platform-agnostic behind `TunnelChannel`
Real payment processing	Provider not yet selected; entitlement plumbing is complete and provider-shaped
Desktop clients (Windows/macOS/Linux)	Not required for the MVP audience
Per-user bandwidth metering and quotas	Not needed at MVP fleet size; no billing dependency on it
Ad-funded free tier (rewarded video for filtering)	Built, then removed on 2026-09-18 — reward was client-asserted and unverifiable, and ad-network domains had to be allowlisted through the very filter being sold
Auto-connect on untrusted Wi-Fi	Built, then removed on 2026-09-18 — required `ACCESS_FINE_LOCATION` on a privacy product and could not work from a cold start under Android's background service rules
Web dashboard / admin console	Operations are runbook-driven at current scale

4. Stakeholders

Stakeholder	Interest
End customer	A working tunnel, a believable privacy claim, a price that is worth it
Product owner	Trial-to-paid conversion, feature scope, app-store compliance
Backend engineering	API correctness, entitlement, control-plane integrity
Mobile engineering	Android client, platform VPN integration, store release
Infrastructure / operations	Node provisioning, fleet health, incident response
Security	Key custody, blast radius, egress policy
Finance	Unit economics per active device, payment provider fees
Legal / compliance	Logging policy, jurisdiction of nodes, app-store and payment rules

5. User personas

P1 — The privacy-motivated individual. Wants their ISP and the sites they visit to see someone else's address. Cares that the provider cannot read the traffic and will read the privacy claims closely. Will notice if the app leaks its real IP.

P2 — The geography-motivated user. Wants to appear to be in another country. Cares about which countries exist, and whether the connection is fast enough to be usable.

P3 — The convenience user. Installs on a phone, wants one button. Will not read documentation, will not manage keys, and will abandon the product at the first configuration screen.

P4 — The fleet operator (internal). Provisions nodes, watches health, rotates credentials, responds when a country goes dark.

---

6. Business requirements

Requirements are stated as business needs. Section 7 traces each to delivered functionality.

6.1 Account and identity

ID	Requirement	Priority
BR-01	A customer must be able to create an account with an email address and a password, and sign in on any device	Must
BR-02	Sessions must persist across app restarts without the customer re-entering credentials	Must
BR-03	A stolen session credential must be detectable and revocable without the customer's involvement	Must
BR-04	Credential endpoints must resist automated guessing and stuffing	Must
BR-05	Deleting an account must remove its devices and its access to the fleet	Should

6.2 Connection

ID	Requirement	Priority
BR-06	A customer must reach a connected state from a single action on the home screen	Must
BR-07	The product must never hold a customer's tunnel private key	Must
BR-08	The customer must be shown evidence that their traffic is actually exiting the chosen node, not merely that the app says "connected"	Must
BR-09	The customer must be able to choose a country, or delegate the choice to the product	Must
BR-10	A delegated choice must be defensible and described honestly to the customer	Must
BR-11	When a tunnel drops unexpectedly, the product must attempt to restore it rather than silently leaving the customer unprotected	Must
BR-12	The customer must be able to exclude named apps from the tunnel	Should
BR-13	The customer must be able to connect and disconnect without opening the app	Could
BR-14	The customer must be able to see their recent session history on the device	Could

6.3 Devices

ID	Requirement	Priority
BR-15	A customer must be able to register multiple devices under one account, up to a published limit	Must
BR-16	A customer must be able to revoke a device, and revocation must remove its access to the fleet immediately	Must
BR-17	The customer must be able to see which of their devices are actually in use	Should
BR-18	Device capacity must be recoverable — a revoked device must not permanently consume a slot	Must

6.4 Fleet and availability

ID	Requirement	Priority
BR-19	The product must support exit nodes in more than one country under one control plane	Must
BR-20	A peer issued for a country must be installed on that country's node, or the request must fail loudly	Must
BR-21	A node that stops answering must be marked unavailable and must not be offered to customers	Must

ID	Requirement	Priority
BR-22	Node capacity must be visible to the product so customers are not routed to a full node	Must
BR-23	Adding a country must be an operational task, not a software release	Should

6.5 Privacy and protection

ID	Requirement	Priority
BR-24	DNS resolution must not leave the tunnel	Must
BR-25	Tunnelled customers must not be able to reach the cloud provider's metadata service or the operator's private networks	Must
BR-26	The fleet must not become a spam relay	Must
BR-27	The product must be able to block advertising and tracker domains for entitled customers, and the customer must be able to turn it off	Should
BR-28	The product must not retain per-customer browsing or query logs	Must

6.6 Commercial

ID	Requirement	Priority
BR-29	A new account must get full functionality free for a bounded trial period	Must
BR-30	The trial must not be restartable by the customer	Must
BR-31	After the trial, paid features must require an active subscription, offered as monthly or yearly	Must
BR-32	Entitlement for any feature the server controls must be enforced on the server	Must
BR-33	Paid access must be granted only on verified payment evidence from the provider	Must — **not yet met**
BR-34	Renewing early must add time rather than discard remaining time	Should

7. Functional requirements as delivered

7.1 Accounts and sessions (satisfies BR-01…BR-04)

- `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, `POST /auth/logout`.
- Passwords are stored as argon2 hashes. Refresh tokens are stored hashed, so a database disclosure does not hand out live sessions.
- Refresh tokens rotate on use and record what replaced them. Presenting an already-rotated token is treated as evidence of theft and revokes the whole token family — verified against a live database.
- Access tokens live 15 minutes; refresh tokens 30 days. The client refreshes transparently through a single Dio interceptor.
- Rate limits: 5/min on register and login, 20/min on refresh, 120/min globally per source address.

7.2 Connection lifecycle (satisfies BR-06…BR-14)

- The Android client generates an X25519 key pair on device using `cryptography`, writes the private key to the Android Keystore via `flutter_secure_storage`, and uploads only the public key. The full peer configuration is displayed exactly once; `GET /devices` deliberately omits the node public key and endpoint so a config cannot be reconstructed later.
- The tunnel itself runs through Android's `VpnService` via `wireguard-android`, driven from Dart across a single `TunnelChannel` platform boundary.
- **Exit verification:** after connecting, the client calls `GET /whoami` and shows the customer the address the internet actually sees, so "connected" is corroborated rather than asserted (BR-08).
- **Location choice:** `GET /nodes` returns the fleet with per-node load and health, and the client renders regions as countries with flags. The customer may pick a country or choose "Closest to you".
- **"Closest to you"** ranks nodes by estimating longitude from the device's UTC offset and comparing against country centroids in one-hour bands, preferring the emptier node within a band. The locations screen states this is an estimate (BR-10). The client sends an explicit `nodeId`; server-side least-loaded selection remains only as the fallback for a request that names no node.
- **Kill switch:** an unexpected drop triggers automatic reconnection, and the screen directs the customer to Android's system-level "Always-on VPN / Block connections without VPN" setting for lockdown that survives the app being killed. A user-requested disconnect is marked as such so it is not rebuilt.
- **Split tunnelling:** a per-app exclusion picker; excluded packages are passed to the tunnel builder.
- **Quick Settings tile:** connect and disconnect from the notification shade, including from a cold start.
- **Session history:** connection records kept on the device only — never uploaded, which is what BR-28 requires.

7.3 Devices (satisfies BR-15…BR-18)

- `POST /devices` checks the cap, selects or accepts a node, allocates the lowest free tunnel address inside that node's subnet, writes the device row, and programs the peer on the correct node's interface. `GET /devices`, `GET /devices/:id/config` and `DELETE /devices/:id` complete the lifecycle.
- Default cap is 5 devices per account (`MAX_DEVICES_PER_USER`, configurable 1–100).
- Revocation is a soft delete (`revokedAt`), so a tunnel address is not recycled while a stale client may still present the old key; revoked devices do not count against the cap (BR-18).
- Concurrent allocation is made safe by a database uniqueness constraint on `(nodeId, tunnelIpV4)` — the allocator relies on the database to reject a double-assignment rather than on application-level locking.
- `Device.lastSeenAt` is advanced from real WireGuard handshake timestamps by a sweep that polls every active node (default every 60 seconds) (BR-17).

7.4 Fleet control plane (satisfies BR-19…BR-23)

- Each node row carries its own region, endpoint, subnet, server public key, resolver addresses, MTU and capacity, so a country is a row plus a provisioned machine (BR-23).
- **Node-scoped programming:** a node is programmed locally only when its id matches `WG_NODE_ID`; any other node must have an agent URL. A node with neither fails with an explicit error rather than silently programming the wrong interface (BR-20). This was

demonstrated end to end on 2026-09-11: a Frankfurt peer appeared on Frankfurt's `wg0` and was absent from Mumbai's, and revocation removed it from Frankfurt.

- **Node agent:** a second entrypoint on the same backend artifact (`node dist/agent/main`), bearer-authenticated, with no database access and no JWT secrets. Compromising a node yields control of that node's interface and nothing more (BO-7).
- **Health and failover:** the API probes each node's control plane and reports `healthy` separately from `available`, so the client can distinguish "full" from "offline" (BR-21, BR-22).
- **Reconciliation:** PostgreSQL is the source of truth for peers, and `wg0` is rebuilt from the database on boot, which makes an exit node disposable and replaceable.

7.5 Protection stack (satisfies BR-24…BR-28)

- Unbound runs on each node so DNS resolution never leaves the tunnel (BR-24).
- The nftables egress chain blocks cloud metadata (`169.254.0.0/16`), RFC1918 destinations, and SMTP (BR-25, BR-26). The first two are load-bearing: without them a customer could reach the instance metadata endpoint and take the cloud credentials.
- Blocky sits in front of Unbound as a filtering resolver (~79,000 domains). Filtering is selected by which **resolver address** the API writes into the customer's configuration, so a modified client cannot grant itself filtering (BR-27, BR-32). Nodes that have not been reprovisioned with an unfiltered listener report the toggle as unsupported rather than lying about it.
- Query logging is off. Session history is on-device only (BR-28).

7.6 Entitlement and monetisation (satisfies BR-29…BR-34)

- **Trial:** 24 hours, derived from `users.createdAt`. There is no "trial started" column, so the client cannot restart its own trial (BR-30).
- **Plans:** monthly (₹149) and yearly (₹1,199, presented as two months free). Plan durations are 30 and 365 days and are written from the plan the customer chose, so the paywall and the database cannot disagree.
- **Entitlement** is trial OR subscription, evaluated at a single instant per request so a profile cannot report "entitled" alongside zero time remaining. `GET /users/me` returns a complete access state — entitled, on trial, subscribed, trial end, subscription end, milliseconds remaining — so the client never does date arithmetic to decide what to show.
- **Early renewal** extends from whichever is later, now or the existing expiry (BR-34).
- **Checkout is a stub.** `POST /users/me/subscription` grants a plan to any authenticated caller. It exists so the paywall can be walked end to end before a provider is chosen, and both the endpoint and the paywall screen say so on their face. **BR-33 is not met and is the single hardest blocker to taking money.**

---

8. Non-functional requirements

ID	Area	Requirement	Status
NFR-01	Key custody	No customer tunnel private key may exist in any server, database, log or backup	Met — structurally
NFR-02	Input safety	Peer mutations must execute without a shell, with an argv array and strict regex validation, since `publicKey` is attacker-controlled	Met
NFR-03	Transport	All client–API traffic over HTTPS (Caddy, automatic certificates); tunnel traffic over WireGuard UDP/51820	Met
NFR-04	Availability	A single node failure must not take the fleet down; customers must be re-routable to another country	Partially met — detection is control-plane only
NFR-05	Recoverability	An exit node must be rebuildable from provisioning scripts plus database state, with no unique local state	Met
NFR-06	Abuse resistance	Rate limiting on all public endpoints, tightest on credential endpoints	Met, with a known flaw (§11 R-4)
NFR-07	Observability	Structured logging with request ids and a global exception filter	Met
NFR-08	Performance	Connect-to-usable under 5 seconds on a normal mobile connection	Met in field use; not formally benchmarked
NFR-09	Latency budget	API calls must not add perceptible delay to app interactions	At risk — database region (§11 R-5)
NFR-10	Portability	The API must be cloud-agnostic; only `ops/` and `docs/` may know the provider	Met
NFR-11	Maintainability	Client features self-contained; `core` never imports a feature	Met
NFR-12	Testability	Unit coverage on the two places where a bug is a security incident — IP allocation and key validation — plus an e2e smoke test	Met
NFR-13	Localisation	Strings externalised for future locales	Structure in place; English only
NFR-14	Store compliance	Permission set must survive Google Play review for a VPN app	Met — `INTERNET` and `ACCESS_NETWORK_STATE` only; location and notification permissions were deliberately removed

9. Data requirements

Entity	Holds	Retention / notes
User	Email, argon2 password hash, created-at, ad-block preference, subscription expiry	Retained for account lifetime; cascade-deletes devices and tokens
RefreshToken	Hashed token, expiry, revoked-at, replaced-by	Pruned opportunistically on login; rotation chain kept for theft detection
Node	Name, region, server public key, endpoint, subnet, resolver addresses, MTU, capacity, active flag, agent URL and token	Operational data, no customer content
Device	Owner, node, name, platform, **public** key, tunnel address, created-at, last-seen-at, revoked-at	Soft-deleted on revoke so addresses are not recycled prematurely

Never stored anywhere in the platform: customer private keys, DNS queries, browsing history, per-session traffic content, connection logs beyond `lastSeenAt`.

Held only on the customer's device: the tunnel private key, the issued configuration, session history, split-tunnel exclusions, kill-switch preference.

10. Assumptions, constraints and dependencies

Assumptions

1. Customers accept an email-and-password account; anonymous or token-only accounts are not required for the MVP.
2. Android is a sufficient beachhead; iOS demand is real but deferrable.
3. The published device cap of 5 matches household usage.
4. Customers will accept a coarse, honestly-labelled “closest” estimate over an exact one.

Constraints

1. **Apple organisation account** — no iOS Network Extension without one. Hard external blocker.
2. **Payment provider not selected** — blocks real revenue and Google Play billing compliance.
3. **WireGuard is silent to unauthenticated packets**, so exit nodes cannot be latency-probed from the client; and geolocating the customer’s address server-side is disqualified by what the product is for. Together these force the time-zone estimate.
4. **Single API instance** — rate limiting and node health are in-memory and per-process; horizontal scaling requires shared state.
5. **Supabase free tier** pauses after ~7 days of inactivity and is currently hosted in Seoul, distant from the Mumbai node.

Dependencies

Dependency	Used for	Risk if it fails
AWS EC2 (Graviton) / Oracle Cloud Ampere	Exit node hosting	Country goes offline
Supabase PostgreSQL	Accounts, devices, nodes	API cannot issue or revoke devices; existing tunnels keep working
WireGuard kernel module, nftables, Unbound, Blocky	The tunnel and its protections	Node-level outage or loss of filtering
Caddy	TLS termination and certificates	API unreachable
`wireguard-android`	Android tunnel implementation	Client cannot connect
Payment provider (TBD)	Revenue	No revenue — current blocker
Apple Developer Program (org)	iOS release	iOS stays deferred

11. Risks

ID	Risk	Impact	Likelihood	Mitigation
R-1	The dummy checkout reaches production and grants free subscriptions to anyone who asks	Total revenue loss	Medium	Replace with provider-webhook-driven grants before any paid release; treat as a release gate
R-2	DNS filtering is bypassable — a client with a hardcoded resolver, or Android Private DNS set to a provider hostname, never reaches Blocky while the UI says filtering is on	Broken promise to a paying customer	High	nftables redirect of tunnel port 53 to the node resolver and rejection of 853; not yet implemented
R-3	Blocky is a single point of failure for DNS on a node — if it does not start, the tunnel comes up and resolves nothing	Country-wide outage that looks like a broken app	Medium	`loading.strategy: fast` degrades to unfiltered rather than dead; rollback is documented; **no alerting exists yet**
R-4	Tunnelled customers share one source address at the API, so the per-IP rate limiter counts a whole node's user base as one caller	Legitimate customers throttled	High once traffic grows	Key the throttler on the authenticated user where one exists
R-5	Every authenticated request performs a user lookup against a database in Seoul; a Mumbai-hosted API pays ~80–120 ms per call	Sluggish app, poor reviews	High at scale	Move the project to a nearer region, or cache the lookup
R-6	Node health only probes the control plane; a node whose agent answers while its data plane is broken is reported healthy	Customers routed to a node that cannot carry traffic	Medium	Today this surfaces only via the client's exit check; add a data-plane probe
R-7	Existing devices do not pick up resolver changes, because DNS is baked into the configuration at issuance	Customers stay on old settings indefinitely	High on any change	Add a configuration-refresh path in the client; decide before changing a node's resolver
R-8	Kill-switch and split-tunnelling entitlements are enforced in the UI only, because the platform performs them locally	Non-paying customers keep paid features on a modified client	Low today, higher with real money	Accept knowingly, or move the gate to configuration issuance
R-9	Google Play review of a VPN application	Launch delay	Medium	Permission surface already minimised to `INTERNET` and `ACCESS_NETWORK_STATE`; privacy claims are technically true
R-10	Abuse from behind the fleet's shared exit addresses (spam, scraping, fraud) leading to blocklisting of node IPs	Service degradation for all customers on that node	Medium	SMTP already blocked at egress; abuse handling process not yet defined
R-11	Several delivered features — split tunnelling, on-device history, the Quick Settings tile — have not been exercised on real hardware for every path	Launch-day defects	Medium	Device test pass before store submission

12. Release gates and acceptance criteria

12.1 Accepted for the current (pre-revenue) release

1. Register, sign in, refresh and sign out against the live API. ✅
2. Issue a device, connect, and confirm via `GET /whoami` that the exit address is the chosen node's. ✅
3. Select a second country and confirm the peer is programmed on that country's interface and nowhere else. ✅
4. Revoke a device and confirm the peer is removed from the correct node. ✅
5. Refresh-token reuse revokes the token family. ✅
6. Ad and tracker filtering resolves `doubleclick.net` to `0.0.0.0` from inside the tunnel while ordinary domains resolve. ✅
7. Permission surface limited to `INTERNET` and `ACCESS_NETWORK_STATE`. ✅

12.2 Gates for a paid public launch

Gate	Requirement	Owner
G-1	Payment provider integrated; access granted only from a signed provider webhook (BR-33, R-1)	Backend
G-2	Google Play Billing compliance for in-app subscriptions	Mobile + Product
G-3	DNS bypass closed at nftables, or the filtering claim softened to match reality (R-2)	Infrastructure
G-4	Alerting on node health, resolver health and certificate expiry (R-3, R-6)	Infrastructure
G-5	Rate limiting keyed on the authenticated user (R-4)	Backend
G-6	Database co-located with the primary node region, or the per-request lookup cached (R-5)	Backend
G-7	Full device test pass on real hardware for every client feature (R-11)	Mobile QA
G-8	Published privacy policy and terms matching what the system actually does	Legal
G-9	Abuse-handling and law-enforcement-request process defined, consistent with holding no logs (R-10)	Legal + Ops

12.3 Deferred roadmap

Item	Trigger
iOS client (Network Extension)	Apple organisation account obtained
Additional countries	Demand, and G-4 alerting in place
City-level node ranking	A fleet with two nodes in one country
Configuration refresh in the client	Before the next resolver or endpoint change (R-7)
Shared-state rate limiting and health (Redis)	Second API instance
"Ads blocked" counter	A decision on how to count without per-customer query logging

13. Traceability

Business requirement	Delivered by	Status
BR-01…BR-04	`POST /auth/*`, argon2, hashed rotating refresh tokens, throttling	✅
BR-05	Cascade delete on `User`	⚠️ No self-service account deletion in the client
BR-06	Connect screen, `TunnelChannel`, `VpnService`	✅
BR-07	On-device X25519 keygen; only the public key is uploaded	✅
BR-08	`GET /whoami` and the connect-screen exit card	✅
BR-09, BR-10	`GET /nodes`, locations screen, `RegionGeo` ranking, “Closest to you”	✅
BR-11	Kill switch with auto-reconnect + system lockdown guidance	✅
BR-12	Split-tunnel picker and excluded-apps store	✅ built, device test pending
BR-13	Quick Settings tile	✅ built, device test pending
BR-14	On-device session history	✅ built, device test pending
BR-15, BR-16, BR-18	`POST/GET/DELETE /devices`, cap check, soft delete	✅
BR-17	Handshake sweep writing `Device.lastSeenAt`	✅
BR-19, BR-20	`WG_NODE_ID`, `WgRunnerRegistry`, node agent, `HttpWgRunner`	✅
BR-21, BR-22	`NodeHealthService`, `load` / `available` / `healthy` in `GET /nodes`	⚠️ Control plane only (R-6)
BR-23	Node row + `provision.sh` + agent unit	✅
BR-24	Unbound on node, resolver address issued in configuration	⚠️ Bypassable (R-2)
BR-25, BR-26	nftables egress chain	✅
BR-27	Blocky, `dns` / `dnsUnfiltered` per node, per-user toggle	✅ on Mumbai; Frankfurt lacks the unfiltered listener
BR-28	No query logging; history on device only	✅
BR-29, BR-30	24h trial derived from `createdAt`	✅
BR-31, BR-34	Paywall, `PLAN_DURATION_MS`, `extendSubscription`	✅

Business requirement	Delivered by	Status
BR-32	Filtering selected by issued resolver address	✅ for ad blocking; UI-only for kill switch and split tunnelling (R-8)
BR-33	—	❌ **Not met — dum**

14. Glossary

Term	Meaning
WireGuard	The VPN protocol and kernel module carrying customer traffic
Peer	One device's entry on a node's WireGuard interface: a public key and an allowed tunnel address
Exit node	The machine customer traffic emerges from; what websites see
Control plane	The API and agents that add and remove peers — never in the traffic path
Node agent	A minimal bearer-authenticated service on each node that programs its own interface; no database, no secrets beyond its own
Tunnel address	The private address (e.g. `10.8.0.2/32`) a device holds inside a node's subnet
Entitlement	Whether an account may use paid features right now — trial or subscription
Blocky / Unbound	The filtering resolver and the recursive resolver running on each node
Kill switch	Behaviour that prevents unprotected traffic when the tunnel drops
Split tunnelling	Excluding named apps from the tunnel

Source of truth for delivery state is `PROGRESS.md`. Where this document and the code disagree, the code is right and this document is stale.